



ECOLE UNIVERSITAIRE DE PHYSIQUE ET
D'INGÉNIERIE

MASTER PAR – 3D DEEP LEARNING
RAPPORT DE PROJET POINTNETLAB

Classification de nuages de points 3D avec PointNet

Réalisé par :
NOCHI MAGOUO

Encadré par :
Paul CHECCHIN

24 février 2026

Table des matières

Resume	1
1 Introduction	2
1.1 Contexte du module	2
1.2 Lien avec l'article PointNet	2
1.3 Objectif de mon rapport	2
2 Methodes et architectures	3
2.1 Idee generale que je retiens de PointNet	3
2.2 Reseau 1 : PointMLP (Exercice 1)	3
2.3 Reseau 2 : PointNetBasic (Exercice 2.1)	4
2.4 Reseau 3 : PointNetFull avec T-Net 3x3 (Exercice 2.2)	5
2.5 Comparaison structurelle des trois reseaux	5
3 Protocole experimental et demarche scientifique	6
3.1 Comment j'ai structure ma demarche	6
3.2 Donnees et constantes de travail	6
3.3 Pipeline de pretraitement et augmentation	7
3.4 Role de chaque bloc de runs	7
3.5 Metriques suivies	7
4 Resultats et visualisations	9
4.1 Visualisation des augmentations point cloud	9
4.2 Campagne C1 : benchmark 10 augmentations (seed 42, 25 epochs)	10
4.3 Campagne C2 : benchmark robuste (seed 1)	11
4.4 Runs finaux baseline + occlusion	13
5 Discussion et prise de decision	14
5.1 Pourquoi je retiens l'occlusion	14
5.2 Ce que j'ai appris des autres essais	14
5.3 Lien avec l'evaluation pedagogique	14
5.4 Limites de ma version actuelle	14
6 Conclusion	15
Bibliographie	16

Table des figures

2.1	Architecture du PointMLP (conforme au notebook : $3072 \rightarrow 512 \rightarrow 256 \rightarrow K$).	4
2.2	Architecture du PointNetBasic (sans T-Net), basee sur les blocs <code>Conv1d</code> du notebook.	5
2.3	Architecture du PointNetFull avec module <code>Tnet</code> d'alignement en entree. . .	5
3.1	Pipeline de pretraitement/augmentation conforme aux classes du notebook.	7
4.1	Galerie des augmentations appliquees aux nuages de points.	9
4.2	Vue globale de la campagne C1.	11
4.3	Courbes de validation de toutes les configurations C1.	11
4.4	Courbes d'apprentissage de la campagne C2.	12
4.5	Matrices de confusion comparees (C2).	12
4.6	Exemple d'un run cible <code>plus_occlusion</code> (C3, seed 0).	13

Liste des tableaux

2.1	Comparaison de mes trois implementations.	5
3.1	Pourquoi j'ai lance chaque type de run.	7
4.1	Intuition metier derriere chaque augmentation.	10
4.2	Resultats C1 (<code>pointnetfull_aug_suite_20260208_145104</code>).	10
4.3	Resultats C2 (<code>pointnetfull_aug_suite_20260211_081948</code>).	11
4.4	Synthese des runs finaux baseline + occlusion.	13

Resume

Dans ce rapport, je montre ma progression sur le projet PointNetLab : implementation des architectures, lecture de l'article PointNet, campagnes d'augmentation, et justification des choix experimentaux. Mon objectif n'est pas seulement d'obtenir un score eleve, mais de justifier une demarche scientifique adaptee au traitement de point clouds.

Chapitre 1

Introduction

1.1 Contexte du module

Dans ce module, je travaille sur le traitement de nuages de points 3D. Un point cloud est un ensemble de points (x, y, z) non ordonnés : je ne peux donc pas supposer un voisinage fixe comme dans une image 2D. Le problème principal est de construire un modèle qui respecte cette structure en ensemble.

1.2 Lien avec l'article PointNet

Le projet est inspiré de l'article *PointNet : Deep Learning on Point Sets for 3D Classification and Segmentation* [1]. Avant d'implémenter mes réseaux, j'ai essayé de comprendre sa logique :

- traiter directement les points, sans voxelisation lourde ;
- apprendre des descripteurs locaux par point via des MLP partagés ;
- utiliser une fonction symétrique (max-pooling) pour obtenir une invariance à l'ordre des points ;
- ajouter un T-Net pour stabiliser l'alignement géométrique en entrée.

1.3 Objectif de mon rapport

Je veux montrer que je comprends ce que je fais, et pas seulement que j'obtiens un score. Concrètement, je documente :

- mon évolution de PointMLP vers PointNet ;
- ma méthode expérimentale (runs, contrôles, comparaison) ;
- pourquoi j'ai retenu **baseline + occlusion** en fin de projet.

Chapitre 2

Methodes et architectures

2.1 Idee generale que je retiens de PointNet

La forme que je retiens de l'article est :

$$f(\{x_1, \dots, x_n\}) \approx \gamma \left(\max_i h(x_i) \right) \quad (2.1)$$

avec h un encodeur partage par point et $\max$ une aggregation symetrique. Cette ecriture m'a servi de fil conducteur pour comprendre pourquoi PointNet fonctionne mieux qu'un MLP classique sur des donnees non ordonnees.

2.2 Reseau 1 : PointMLP (Exercice 1)

J'ai d'abord implemente le modele demande dans l'enonce : flatten 1024×3 puis couches fully-connected.

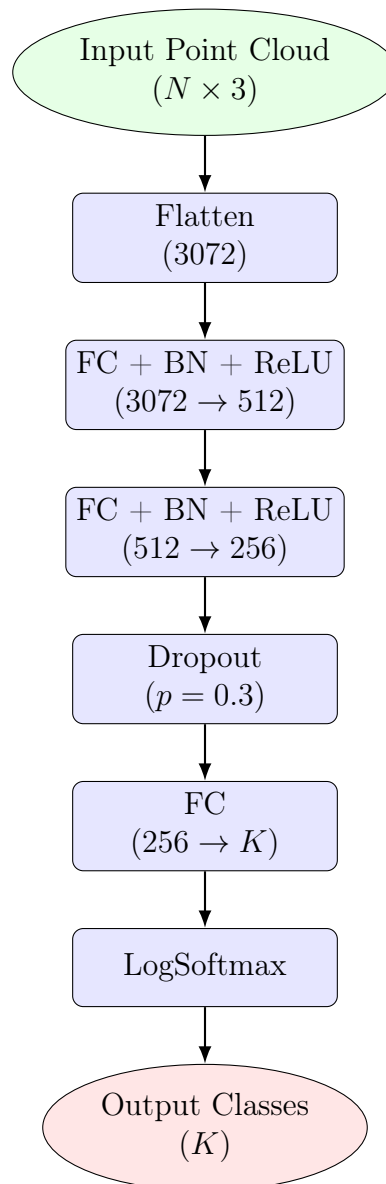


FIGURE 2.1 – Architecture du *PointMLP* (conforme au notebook : $3072 \rightarrow 512 \rightarrow 256 \rightarrow K$).

Analyse. Avec ce modele, je perds la structure "ensemble de points" au moment du flatten. Le modele voit un grand vecteur ordonne, alors que l'ordre des points est arbitraire. C'est la limite qui m'a pousse a passer a PointNetBasic.

2.3 Reseau 2 : PointNetBasic (Exercice 2.1)

Dans PointNetBasic, je remplace le flatten par des MLP partages (Conv1d de noyau 1) puis un max-pooling global.

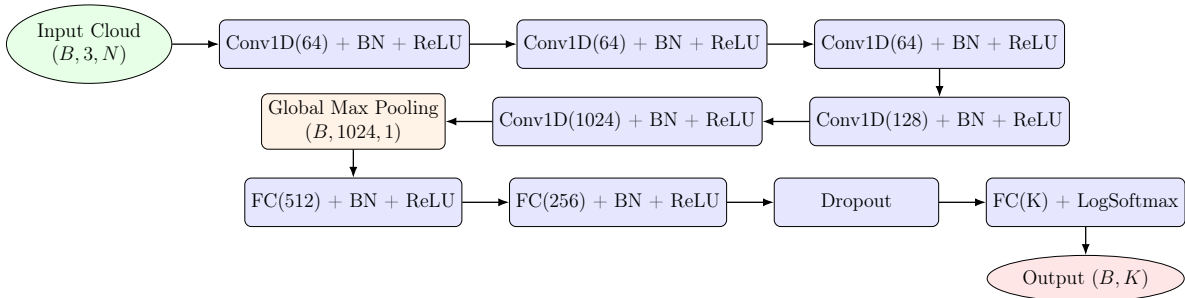


FIGURE 2.2 – Architecture du *PointNetBasic* (sans *T-Net*), basée sur les blocs *Conv1d* du notebook.

Analyse. Ce schéma corrige le problème principal de PointMLP : le max-pooling rend la sortie invariante à la permutation des points. Pour le traitement de point clouds, c'est une différence fondamentale.

2.4 Réseau 3 : PointNetFull avec T-Net 3x3 (Exercice 2.2)

La version Full ajoute un T-Net d'alignement 3x3 en entrée. J'utilise aussi une régularisation orthogonale :

$$\mathcal{L} = \mathcal{L}_{CE} + \lambda \|I - AA^\top\|_F^2 \quad (2.2)$$

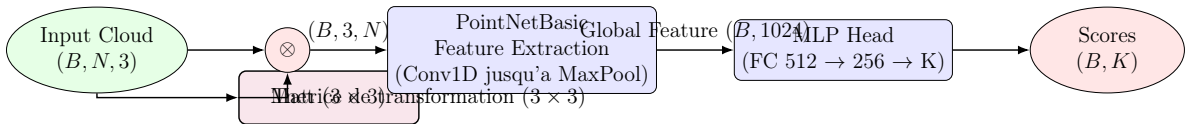


FIGURE 2.3 – Architecture du *PointNetFull* avec module *Tnet* d'alignement en entrée.

Analyse. Le T-Net apporte une adaptation géométrique supplémentaire. Dans ma lecture, il agit comme une normalisation apprenable : le réseau apprend à "mettre en forme" le nuage avant extraction des features globales.

2.5 Comparaison structurelle des trois réseaux

TABLE 2.1 – Comparaison de mes trois implementations.

Modele	Invariance ordre	Alignement geometrique	Params trainables
PointMLP	Non	Non	1,708,810
PointNetBasic	Oui (max-pooling)	Non	811,914
PointNetFull	Oui (max-pooling)	Oui (T-Net 3x3)	1,614,995

Analyse. J'insiste sur un point : l'amélioration ne vient pas simplement d'avoir plus de paramètres. Elle vient surtout du fait que PointNetBasic et PointNetFull respectent mieux la nature d'un point cloud (ensemble non ordonné, variations géométriques).

Chapitre 3

Protocole experimental et demarche scientifique

3.1 Comment j'ai structure ma demarche

Le professeur a insiste sur la demarche scientifique ; j'ai donc structure mes essais comme suit :

1. je pose une hypothese (ex : l'occlusion doit mieux simuler les cas reels de points manquants) ;
2. je fais varier un facteur principal a la fois ;
3. je garde des constantes (dataset, architecture de reference, batch size, metriques) ;
4. j'analyse a la fois les chiffres et les visuels (courbes, matrices de confusion, nuages augmentes) ;
5. je ne retiens une decision que si elle reste defendable sur plusieurs runs.

3.2 Donnees et constantes de travail

Dans mon notebook final, j'utilise :

- ModelNet10_PLY (10 classes, 1024 points/objet),
- split : train 3193 / val 798 / test 908,
- PointNetFull comme modele de reference pour les campagnes d'augmentation,
- scheduler actif et early stopping pilote sur la validation,
- test execute en fin de run seulement.

3.3 Pipeline de pretraitement et augmentation

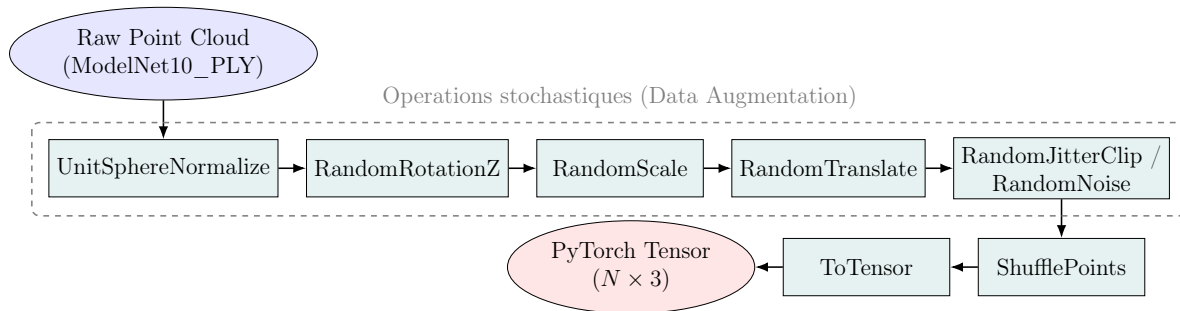


FIGURE 3.1 – Pipeline de pretraitement/augmentation conforme aux classes du notebook.

Analyse. Cette representation m’aide a montrer que les augmentations ne sont pas appliquees "au hasard" : je normalise d’abord la geometrie globale, puis j’injecte des variations stochastiques realistes (RandomRotationZ, RandomScale, RandomTranslate, RandomJitterClip/RandomNoise) avant conversion en tenseur. Cette logique est coherente avec le traitement de point clouds et guide l’interpretation des resultats de chaque run.

3.4 Role de chaque bloc de runs

TABLE 3.1 – Pourquoi j’ai lance chaque type de run.

Bloc	Configurations	Question a laquelle je veux repondre
Baseline controlee	baseline (rot_z + bruit)	Quel est mon niveau de base ?
Ablations unitaires	+jitter, +mirror, +scale, +translate, +dropout, +occlusion	Quelle augmentation apporte un vrai gain seule ?
Combinaisons	combo_geo, combo_robust, combo_full	Les gains se cumulent-ils reellement ?
Benchmark robuste	baseline, +jitter, +occlusion, combo_full	Le classement reste-t-il stable sous autre protocole ?
Fine-tuning final	baseline + occlusion (seed 42, 45, puis nouveau seed)	Le choix final tient-il au test final ?

Analyse. Ce tableau resume ma logique de validation : je passe d’abord par des ablations, puis je verifie la robustesse avant de figer le choix final.

3.5 Metriques suivies

- `best_val_acc` : meilleure accuracy validation pendant l’entrainement ;
- `final_val_acc` : accuracy validation au dernier epoch execute ;

- `best_eval_acc` : score du checkpoint selectionne ;
- `final test accuracy` : mesure finale sur test en fin de run.

Analyse. Cette separation m'évite de confondre une bonne dynamique de validation avec une vraie generalisation sur test.

Chapitre 4

Resultats et visualisations

4.1 Visualisation des augmentations point cloud



FIGURE 4.1 – *Galerie des augmentations appliquees aux nuages de points.*

Commentaire. Cette figure m’a aide a verifier que les transformations restent physiquement plausibles. En particulier, l’occlusion supprime localement une zone coherente du nuage, ce qui colle mieux aux cas reels qu’un bruit purement aleatoire.

TABLE 4.1 – *Intuition metier derriere chaque augmentation.*

Augmentation	Intuition point cloud	Ce que j’observe dans mes runs
<code>jitter</code>	simuler un bruit capteur local	bon score ponctuel, stabilite moins bonne selon protocole
<code>mirror</code>	tester des symetries geometriques	utile, mais effet tres dependant des classes
<code>scale/translate</code>	variabilite de pose et distance	gain modere seul, plus utile en combinaison
<code>dropout</code>	simuler des points manquants aleatoires	robustesse partielle, gain limite
<code>occlusion</code>	simuler un masquage local realiste	meilleur compromis robustesse / performance finale

Commentaire. Cette table resume pourquoi je ne choisis pas l’augmentation uniquement sur un classement de score, mais sur sa coherence avec les problemes typiques d’acquisition 3D.

4.2 Campagne C1 : benchmark 10 augmentations (seed 42, 25 epochs)

TABLE 4.2 – *Resultats C1 (pointnetfull_aug_suite_20260208_145104).*

Configuration	best_val_acc (%)	best_epoch	final_val_acc (%)
plus_jitter	91.08	23	86.67
plus_mirror	90.75	21	87.56
plus_occlusion	90.42	23	89.76
baseline	90.31	23	88.77
plus_scale	90.31	21	87.33
plus_translate	90.31	21	87.56
combo_robust	90.20	23	86.23
combo_full	90.09	22	90.09
plus_dropout	89.65	23	89.43
combo_geo	88.33	22	86.78

Commentaire. Jitter arrive premier en `best_val_acc` dans cette campagne, mais je remarque deja que `plus_occlusion` donne une meilleure `final_val_acc` que la baseline avec un comportement plus regulier. C’est un premier signal en faveur de l’occlusion.

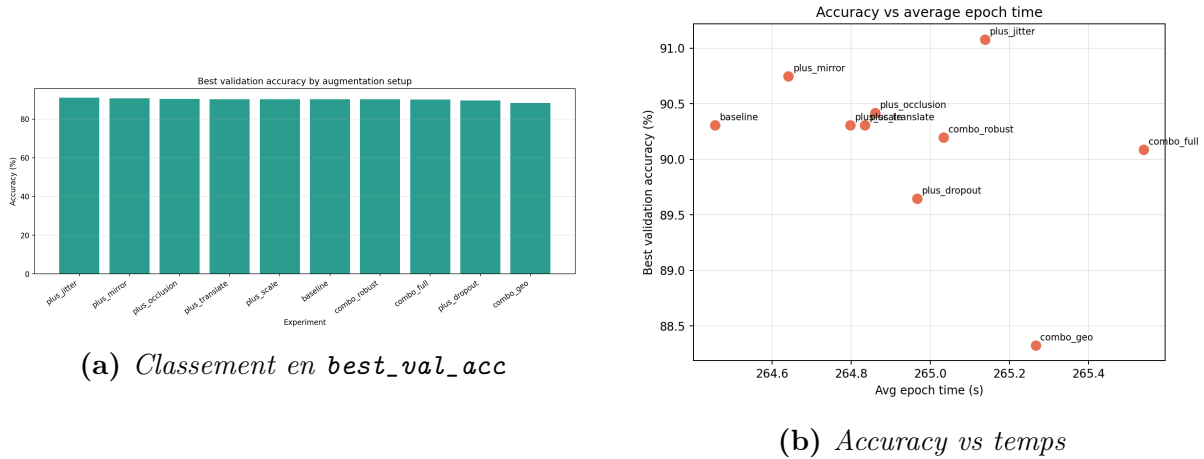


FIGURE 4.2 – Vue globale de la campagne C1.

Commentaire. Je lis ces graphes ensemble : je ne cherche pas seulement le meilleur score, je regarde aussi le compromis temps/qualite. Une config qui "gagne" de quelques dixiemes mais devient instable n'est pas forcément la meilleure decision experimentale.

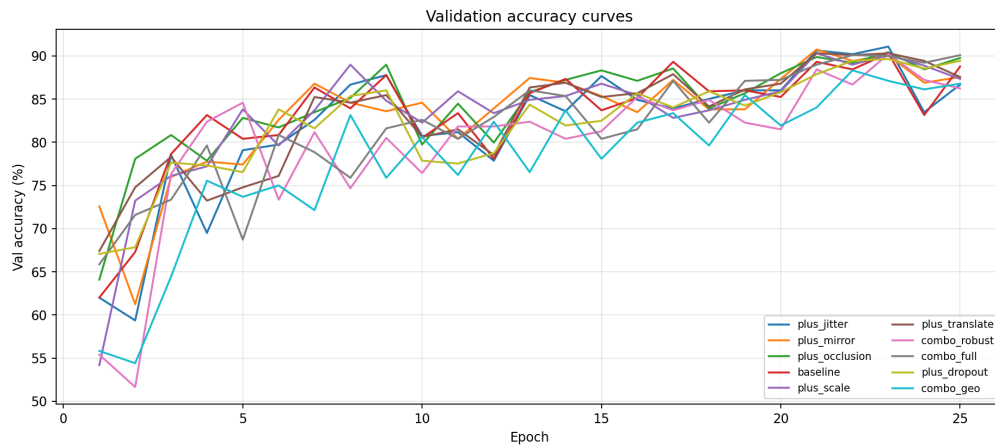


FIGURE 4.3 – Courbes de validation de toutes les configurations C1.

Commentaire. Cette figure confirme que plusieurs runs montent vite puis oscillent. L'occlusion fait partie des courbes que je trouve les plus exploitables pour un fine-tuning final.

4.3 Campagne C2 : benchmark robuste (seed 1)

TABLE 4.3 – Resultats C2 (*pointnetfull_aug_suite_20260211_081948*).

Configuration	best_val_acc (%)	best_epoch	final_val_acc (%)	epochs_ran
plus_occlusion	88.33	21	87.33	27
combo_full	88.22	18	88.00	24
baseline	86.45	10	81.61	16
plus_jitter	86.12	7	83.81	13

Commentaire. Ici, l'occlusion passe clairement devant. C'est important pour ma décision : le classement ne repose plus sur une seule campagne, il se repete dans un protocole different.

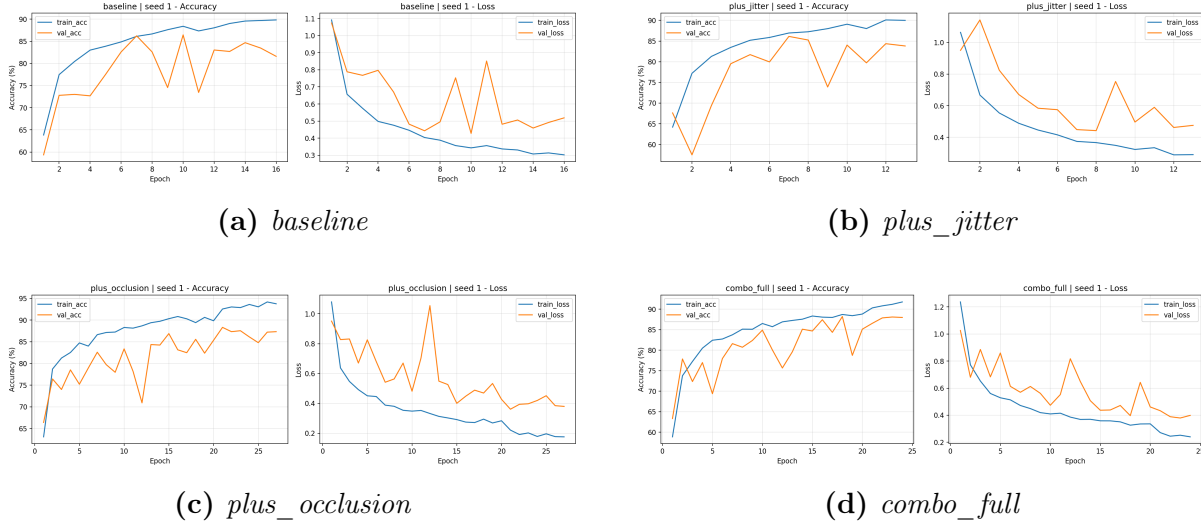


FIGURE 4.4 – Courbes d'apprentissage de la campagne C2.

Commentaire. Je vois que les configurations plus complexes ne sont pas systematique-ment meilleures. Le signal utile pour moi est la regularite de progression et la capacite a conserver un bon niveau en validation.

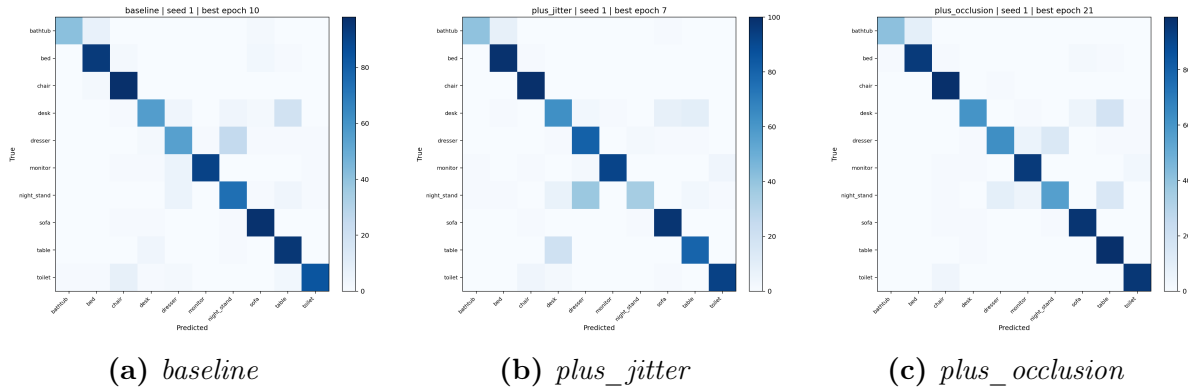


FIGURE 4.5 – Matrices de confusion comparees (C2).

Commentaire. Les matrices me permettent de voir quelles classes restent fragiles. J'utilise cette lecture qualitative pour verifier que le gain de l'occlusion n'est pas seulement global, mais qu'il se retrouve aussi dans la repartition des erreurs.

4.4 Runs finaux baseline + occlusion

TABLE 4.4 – *Synthese des runs finaux baseline + occlusion.*

Run notebook	max val_acc (%)	final test_acc (%)	commentaire
seed 42 (archive)	94.36	90.09	reference initiale
seed 45 (archive)	95.11	90.75	progression confirmee
seed 50 (nouveau test)	—	—	run execute, export des metriques en

Commentaire. Je garde une trace explicite des seeds pour montrer que je ne me base pas sur un unique run "chanceux". Le run supplementaire seed 50 est deja lance ; il ne manque que l'export final des artefacts pour renseigner ses chiffres dans ce tableau.

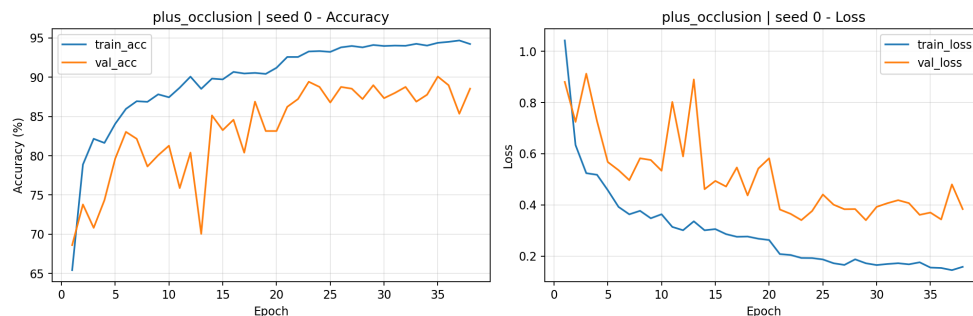


FIGURE 4.6 – *Exemple d'un run cible plus_occlusion (C3, seed 0).*

Commentaire. Je conserve aussi cette courbe detaillee pour illustrer la dynamique d'entrainement d'un run long, et justifier le choix de checkpoint retenu.

Chapitre 5

Discussion et prise de decision

5.1 Pourquoi je retiens l’occlusion

Mon choix final **baseline + occlusion** repose sur trois elements :

- **coherence avec le traitement de point clouds** : les points manquants en zone localisee sont un cas realiste en acquisition 3D ;
- **stabilite entre campagnes** : l’occlusion reste bien classee en C1 et arrive premiere en C2 ;
- **impact final sur test** : les runs archives montrent une progression nette (90.09% puis 90.75%), et le run seed 50 est ajoute pour confirmer cette tendance.

5.2 Ce que j’ai appris des autres essais

- **jitter** peut etre excellent ponctuellement, mais moins stable selon les protocoles ;
- les combinaisons lourdes ne donnent pas automatiquement un gain ;
- **dropout**, **scale** et **translate** sont utiles pour diversifier, mais n’ont pas suffi seuls pour battre durablement l’occlusion.

5.3 Lien avec l’évaluation pedagogique

Comme le professeur l’a precise, la note ne depend pas uniquement du score maximal. J’ai donc essaye de montrer une logique complete : hypothese, protocole, comparaison, puis justification argumentee du choix final.

5.4 Limites de ma version actuelle

- je dois encore consolider un vrai tableau $mean \pm std$ sur plusieurs seeds pour toutes les configs ;
- je n’ai pas encore archive quantitativement de facon homogene toute la transition PointMLP $\rightarrow$ PointNetBasic $\rightarrow$ PointNetFull sur une meme grille d’epochs ;
- je reste focalise sur ModelNet10, alors qu’une extension ModelNet40 serait interessante pour generaliser la conclusion.

Chapitre 6

Conclusion

Dans ce travail, j'ai essaye de montrer a la fois les resultats et la logique qui m'y conduit. J'ai implemente les trois etapes demandees (PointMLP, PointNetBasic, PointNetFull), puis j'ai construit une comparaison experimentale pour choisir une politique d'augmentation defendable.

La conclusion que je retiens est la suivante : pour mon contexte de point clouds ModelNet10, l'occlusion est l'augmentation la plus pertinente pour un bon compromis robustesse / performance.

La suite immediate est claire :

- finaliser l'export du run seed 50 et renseigner ses metriques dans les tableaux,
- etendre la comparaison multi-seeds en $mean \pm std$,
- puis figer une version finale courte et propre pour la soumission.

Bibliographie

- [1] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. Pointnet : Deep learning on point sets for 3d classification and segmentation. In *CVPR*, 2017.